DRDO Research Internship Report

Surrogate Neural Network Guided Evolutionary Optimization for the P-Centre Problem

Submitted by

Deevi Venkata Naga Sai Yaswanth

Under the expert mentorship of:

V. Pavan Kumar

ABSTRACT

This work presents a hybrid metaheuristic optimization framework integrating an evolutionary strategy with a neural network surrogate model for solving the P-Center Problem. The algorithm employs rank-based probabilistic selection to preserve population diversity and mitigate premature convergence. A self-adaptive Gaussian mutation mechanism enables individual candidates to dynamically adjust their mutation strength, enhancing search efficiency. Training data collected during evolutionary runs, consisting of normalized candidate features and associated fitness values, are used to train a deep neural network surrogate that approximates the computationally expensive fitness evaluation function. This surrogate enables rapid fitness prediction during subsequent optimization runs, significantly reducing computational cost. Experimental evaluation shows that the hybrid approach achieves solution quality comparable or superior to baseline methods with substantial runtime reductions. These results demonstrate the efficacy of coupling adaptive evolutionary search with surrogate modelling for scalable and robust P-Center Problem optimization.

INTRODUCTION

Combinatorial optimization problems are central to operations research and have important applications in logistics, network design, and resource allocation. Among these, the P-Center Problem, which involves selecting a subset of facilities to minimize the maximum distance any client must travel, presents significant computational challenges due to its combinatorial complexity.

Metaheuristic algorithms have been widely adopted to provide efficient approximate solutions for such problems, thanks to their ability to explore large solution spaces without exhaustive search. However, these methods often require extensive fitness evaluations, which can be computationally expensive, particularly when the fitness function is complex.

In this study, we propose a hybrid optimization framework that combines an evolutionary strategy with a neural network surrogate model to address these challenges. The evolutionary strategy includes a rank-based probabilistic selection to maintain diversity within the population and employs a self-adaptive Gaussian mutation to dynamically adjust the search step size for each individual candidate. The neural network surrogate is trained on historical fitness data obtained during optimization runs, enabling fast approximation of fitness evaluations in subsequent iterations.

This approach aims to balance the trade-off between computational efficiency and solution quality, making it scalable for large problem instances. The proposed method is evaluated on instances of the P-Center Problem, demonstrating its capability to find high-quality solutions efficiently.

LITERATURE REVIEW

The P-Center Problem, a classical combinatorial optimization challenge, has received significant attention in operations research due to its applicability in facility location and network design. Multiple variants and formulations of the problem have been studied extensively in terms of theoretical complexity, exact approaches, and heuristic solutions. Early exact methods primarily relied on integer and mixed-integer linear programming formulations and branch-and-bound algorithms [Calik et al., 2013; Daskin, 2013]. While exact methods guarantee optimality, their applicability is limited to small and medium-sized instances due to computational complexity.

To address scalability and computational limitations, a rich body of work has focused on metaheuristic algorithms. These approaches, including genetic algorithms, tabu search, variable neighborhood search, and memetic algorithms, provide approximate solutions by iteratively exploring the solution space using stochastic and adaptive mechanisms [Mladenović et al., 2003; Pullan, 2008]. Metaheuristics are particularly effective for large-scale problems but rely heavily on the number of fitness evaluations, which often dominate overall computational cost.

Increasingly, surrogate modeling techniques have been combined with metaheuristic optimization to alleviate expensive evaluations. Neural networks, due to their ability to learn complex nonlinear mappings, have been widely utilized as surrogate models to approximate fitness functions [Jones, 1998; Sacks et al., 1989]. Surrogate-assisted evolutionary algorithms leverage these models to guide the search process efficiently by reducing the frequency of costly direct evaluations. Various strategies for integrating surrogates, including periodically retraining models with evolving data, have demonstrated improved convergence speed without significant degradation in solution accuracy [Jin, 2011; Forrester et al., 2008].

Recent works have explored self-adaptive mutation mechanisms within evolutionary strategies to dynamically balance exploration and exploitation during the optimization process [Beyer and

Schwefel, 2002]. Similarly, rank-based selection schemes have been proposed to maintain population diversity and prevent premature convergence by probabilistically selecting candidate solutions relative to their rank rather than raw fitness values [Eiben and Smith, 2015].

This report builds upon these strands of research by proposing a hybrid metaheuristic framework that integrates rank-based selection and self-adaptive Gaussian mutation within an evolutionary strategy, augmented by a neural network surrogate model. This combination aims to efficiently solve large-scale P-Center Problem instances, reducing computational overhead while maintaining high solution quality. Comparative evaluation elucidates the advantages of this integrated approach over conventional methods.

PROPOSED METHOD

PROBLEM SETUP AND INPUTS

In this work, we address the discrete P-Center Problem, where the goal is to select p centers from n candidate nodes to minimize the maximum distance any node has to its closest center. The problem input consists primarily of:

- *Number of nodes (n): Total locations in the problem space.*
- *Number of centers (p): Number of centers to be selected.*
- *Distance matrix (D): An $n \times n$ matrix where each element d_{ij} corresponds to the distance between node i and node j.*

The fitness or quality of any candidate solution (a set of p centers) is measured by:

$$\text{fitness} = \max_{i=1}^{n} \min_{c \in S} d_{i,c}$$

where $S \subseteq \{1, \dots, n\}$ is the candidate set of centers.

The problem can be formally formulated as the integer program:

$$\begin{aligned} \min \quad & Z \\ \text{subject to} \quad & \sum_{j=1}^{n} x_{ij} = 1, \quad \forall i = 1, \dots, n \\ & \sum_{j=1}^{n} y_j = p \\ & x_{ij} \le y_j, \quad \forall i, j \\ & \sum_{j=1}^{n} d_{ij}\, x_{ij} \le Z, \quad \forall i \\ & x_{ij} \in \{0,1\}, \quad y_j \in \{0,1\} \end{aligned}$$

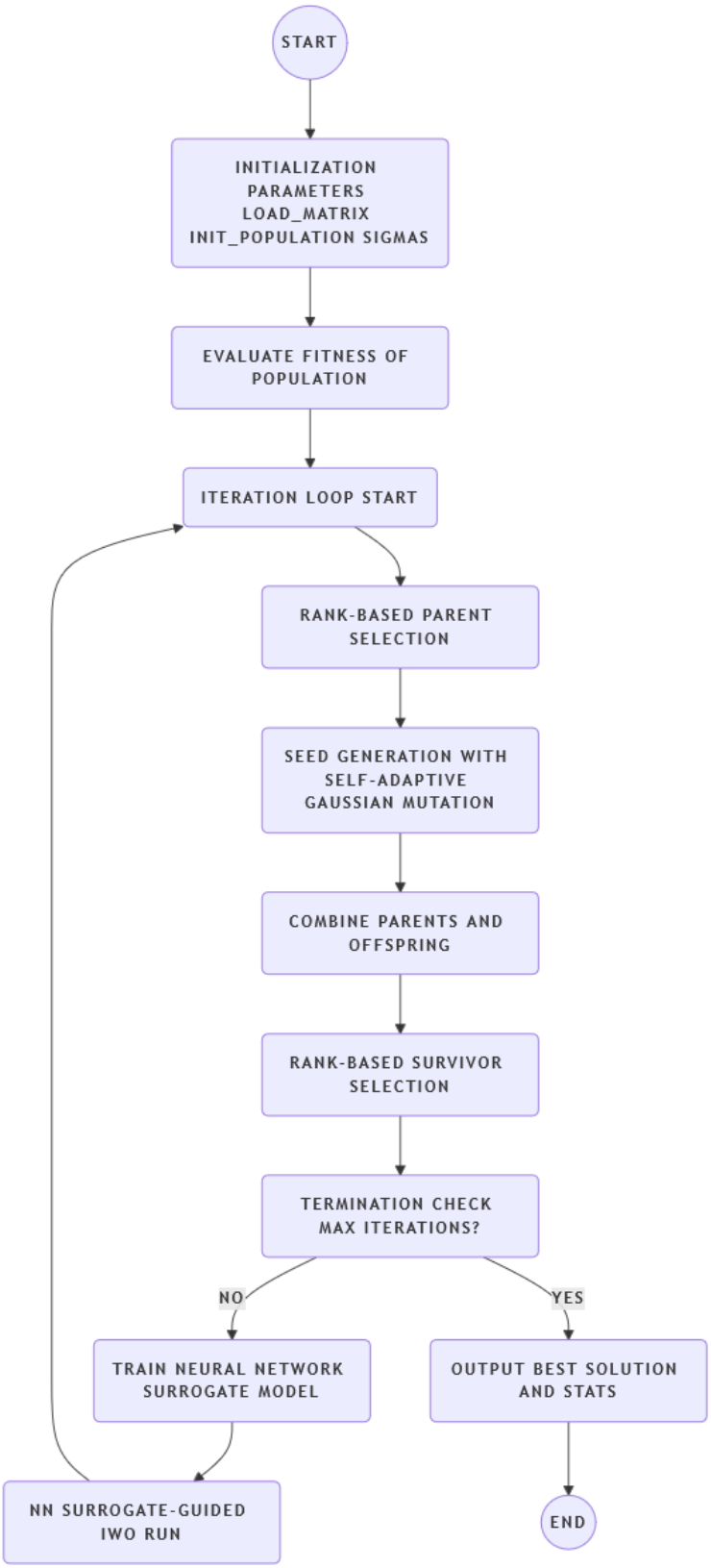
Where,

- $y_j = 1$ *if center is chosen at node j, else 0.*
- $x_{ij} = 1$ *if node i is assigned to center j, else 0.*
- Z *is the minimized maximum assignment distance (covering radius).*

MATRIX GENERATION

The distance matrix D is generated by computing pairwise Euclidean distances between nodes or using preexisting problem-specific distances. This matrix is loaded into the system to enable rapid fitness evaluation of candidate center sets by directly querying distances, avoiding repeated expensive distance computations during optimization.

ALGORITHM OVERVIEW



Algorithm flow

The proposed algorithm solves the P-Center Problem by combining an evolutionary search strategy with a neural network surrogate model to optimize computational efficiency and solution quality.

I) Initialization

- *Define an initial population $P^{(0)} = \{S_1^{(0)}, S_2^{(0)}, \dots, S_{n_i}^{(0)}\}$ of candidate solutions, where each candidate $S_i^{(0)} \subseteq \{1, \dots, n\}$ is a set of p centers selected randomly.*
- *Initialize associated mutation strengths $\sigma_i^{(0)}$ for each candidate solution, controlling the variance in offspring generation.*
- *Load the distance matrix $D = [d_{ij}]_{n \times n}$ for efficient fitness evaluations.*

II) Evolutionary Search

For each generation $t = 1,2, \dots, T$, repeat:

- *Rank-Based Selection: Candidates in $P^{(t-1)}$ are ranked by their fitness values $f\left(S_i^{(t-1)}\right)$. A probabilistic selection mechanism based on rank chooses parent candidates to promote diversity and reduce premature convergence.*
- *Self-Adaptive Gaussian Mutation: From each selected parent $S_i^{(t-1)}$, generate offspring (seeds) by perturbing center indices using Gaussian mutation with self-adapted mutation strengths $\sigma_i^{(t-1)}$:*

$$\sigma_i^{(t)} = \sigma_i^{(t-1)} \times \exp\left(\tau \cdot N(0,1)\right)$$

where,

τ is a learning parameter and $N(0,1)$ is a standard normal random variable. The offspring solution S'_i is obtained by perturbing parent centers according to $\sigma_i^{(t)}$.

- *Fitness Evaluation: Evaluate offspring fitness via:*

$$f(S) = \max_{i=1}^{n} \min_{c \in S} d_{\,i,c}$$

- *Population Update: Combine parents and offspring, employing rank-based survivor selection to form the next generation $P^{(t)}$.*

III) Neural Network Surrogate Model

- *During the evolutionary search, training data consisting of candidate features and their true fitness values are collected.*
- *A deep neural network $\mathcal{N}$ is trained to approximate the fitness function f, enabling predictive evaluation:*

$$\hat{f}(S) = \mathcal{N}\left(\phi(S)\right)$$

where $\phi(S)$ is the feature representation of solution S.

IV) Surrogate-Guided Optimization

- *In later generations or subsequent runs, neural network $\mathcal{N}$ predicts fitness for offspring:*

$$f(S) \approx \hat{f}(S)$$

- *This surrogate evaluation significantly reduces computational overhead compared to explicit fitness calculations.*

EVOLUTIONARY ALGORITHM DETAILS

i)Population Initialization

The initial population $P^{(0)} = \{S_1^{(0)}, S_2^{(0)}, \dots, S_{n_i}^{(0)}\}$ *consists of* n_i *candidate solutions. Each candidate solution* $S_i^{(0)} \subseteq \{1, \dots, n\}$ *is randomly generated by selecting* p *unique centers for the P-Center Problem:*

$$S_i^{(0)} = \text{sorted}(\text{random_sample}(\{1, \dots, n\}, p))$$

This random initialization ensures diverse exploration at the start of the optimization.

ii)Fitness Calculation

The fitness function evaluates the quality of candidate S_i *as the maximum distance from any node to its nearest selected center. Given the distance matrix* $D = [d_{ij}]$, *fitness is computed by:*

$$f(S_i) = \max_{1 \le k \le n} \min_{c \in S_i} d_{k,c}$$

Lower fitness corresponds to better coverage of the nodes.

iii)Rank-Based Selection

Candidates are ranked in ascending order by their fitness values. The probability p_i *of selecting candidate* S_i *as a parent for the next generation is proportional to its rank* r_i*:*

$$p_i = \frac{r_i}{\sum_{j=1}^{n_i} r_j}$$

where the best candidate has the highest rank $r = n_i$*. This probabilistic scheme promotes diversity and guards against the premature convergence of the population.*

iv)Self-Adaptive Gaussian Mutation

Each selected parent $S_i^{(t)}$ *produces offspring through Gaussian mutation with self-adaptive mutation strength. The mutation strength* $\sigma_i^{(t)}$ *evolves according to:*

$$\sigma_i^{(t+1)} = \sigma_i^{(t)} \cdot \exp\left(\tau \cdot N(0,1)\right)$$

where τ *is a learning rate parameter and* $N(0,1)$ *is a standard normal random variable. Offspring* S_i' *are generated by perturbing each center* $c \in S_i$ *as:*

$$c' = \text{clip}\left(\text{round}\left(c + \mathcal{N}\left(0, \sigma_i^{(t+1)}\right)\right), 1, n\right)$$

ensuring valid node indices and uniqueness of centers. This mutation dynamically tunes search step sizes, balancing local exploitation and global exploration.

V)Survivor Selection

Parents and offspring are combined into a temporary population. Rank-based selection is again applied to select the $n_{\max}$ fittest solutions to form the next generation $P^{(t+1)}$, accompanied by their updated mutation strengths.

NEURAL NETWORK SURROGATE

During the evolutionary optimization process, a dataset $\mathcal{D} = \{(\phi(S_i), f(S_i))\}$ is collected, where each sample consists of a feature vector $\phi(S_i)$ representing a candidate solution S_i and its corresponding true fitness value $f(S_i)$.

The surrogate model is a deep neural network $\mathcal{N}$ trained to approximate the fitness function:

$$\hat{f}(S) = \mathcal{N}(\phi(S); \theta)$$

where θ denotes the neural network parameters optimized by minimizing a loss function, typically the mean squared error (MSE):

$$\mathcal{L}(\theta) = \frac{1}{|\mathcal{D}|} \sum_{(\phi(S_i), f(S_i)) \in \mathcal{D}} \left(f(S_i) - \hat{f}(S_i) \right)^2$$

Training occurs periodically during the evolutionary runs to update the neural network parameters with the most recent data, improving the approximation of the fitness landscape.

Once trained, the surrogate model is used to predict fitness values for newly generated candidate solutions during subsequent optimization iterations. This enables rapid evaluation without invoking the computationally expensive direct fitness function:

$$f(S) \approx \hat{f}(S)$$

By substituting true fitness evaluations with surrogate predictions, the algorithm reduces runtime significantly while maintaining solution accuracy.

ITERATION AND TERMINATION

The proposed algorithm proceeds iteratively over a fixed number of generations T. During each generation, the population of candidate solutions undergoes the processes of selection, mutation, and evaluation as described previously.

After executing the specified T iterations, the training phase for the neural network surrogate is completed using the accumulated dataset of candidate features and their corresponding true fitness values.

Subsequently, the algorithm enters the surrogate-guided optimization phase. In this phase, the surrogate model $\mathcal{N}$ replaces the computationally expensive direct fitness evaluations with rapid neural network predictions for offspring candidates:

$$f(S) \approx \hat{f}(S) = \mathcal{N}(\phi(S))$$

This substitution significantly reduces evaluation time per generation, enabling the algorithm to explore more candidate solutions within the same computational budget.

The iterative process continues until either the maximum number of generations T is reached or a convergence criterion is satisfied (e.g., no improvement over successive generations).

The combination of the iterative evolutionary framework with surrogate-assisted evaluation ensures a balance between exploration of the solution space and computational efficiency, ultimately producing high-quality solutions to the P-Center Problem within reasonable time.

SIMULATION

N	*P*	*IWO (Time)*	*Solution*	*Fitness (IWO)*	*Genetic NN Algorithm*	*Solution*	*Fitness*	*Model Training Time*	*MSE*	*MAE*	*R2*
10	3	0.05	[3, 5, 7]	129	1.5139	[0, 2, 8]	141	10.84	260.772	8.7097	0.1165
100	3	0.3321	[35, 37, 94]	159	1.614	[28, 74, 93]	170	11.25	254.7542	10.4011	0.4594
100	10	0.601	[1, 7, 15, 17, 27, 28, 34, 82, 83, 90]	90	1.6288	[5, 16, 20, 27, 39, 46, 48, 65, 81, 89]	111	9.38	73.371	4.5653	0.6155
500	3	1.9702	[48, 277, 456]	191	1.4439	[157, 247, 248]	234	11.89	104.4043	5.4094	0.4698
500	10	4.2	[14, 15, 39, 47, 212, 281, 301, 345, 376, 452]	120	1.6053	[71, 167, 204, 241, 259, 399, 404, 420, 430, 445]	144	10.31	36.5459	3.0496	0.5134
500	100	22.2501	[...]	40	3.4952	[...]	48	9.39	50.2344	3.1999	0.6032
1000	3	3.8573	[253, 277, 768]	203	0.6021	[2, 316, 576]	216	15.75	150.6189	9.0936	0.6681
1000	10	12.2832	[162, 165, 220, 282, 344, 434, 710, 778, 824, 983]	129	0.7191	[21, 27, 104, 185, 371, 414, 457, 530, 702, 886]	153	11.33	54.084	4.5205	0.6025
1000	100	67.9388	[...]	43	3.538	[...]	50	9.22	11.8904	1.9424	0.4858
3000	3	12.0505	[913, 1005, 1126]	216	1.6679	[919, 1585, 2014]	233	14.29	51.3788	4.5123	0.6491
3000	10	38.9923	[76, 375, 396, 424, 1084, 1371, 1407, 1470, 1558, 2474]	139	1.73	[18, 497, 713, 748, 1394, 2356, 2519, 2625, 2710, 2876]	157	15.16	53.9585	4.7819	0.5872
3000	100	371.0341	[...]	49	3.1064	[...]	63	8.74	14.5414	2.3779	0.5764

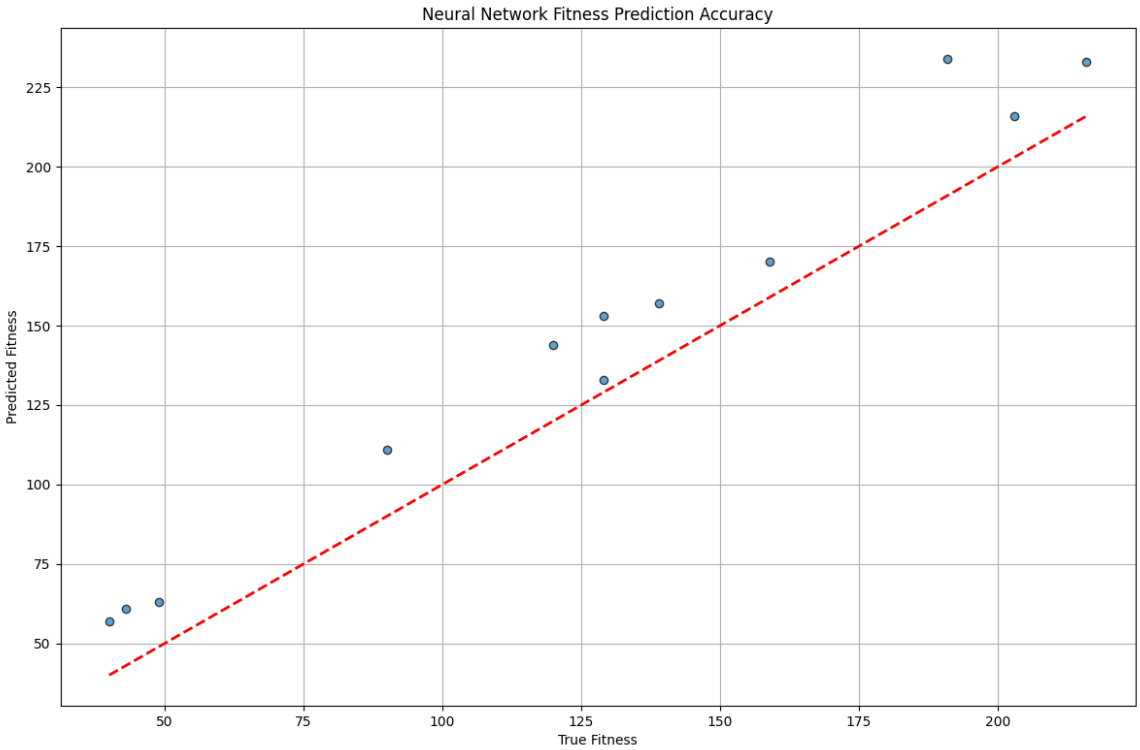
BACKGROUND ON IWO:

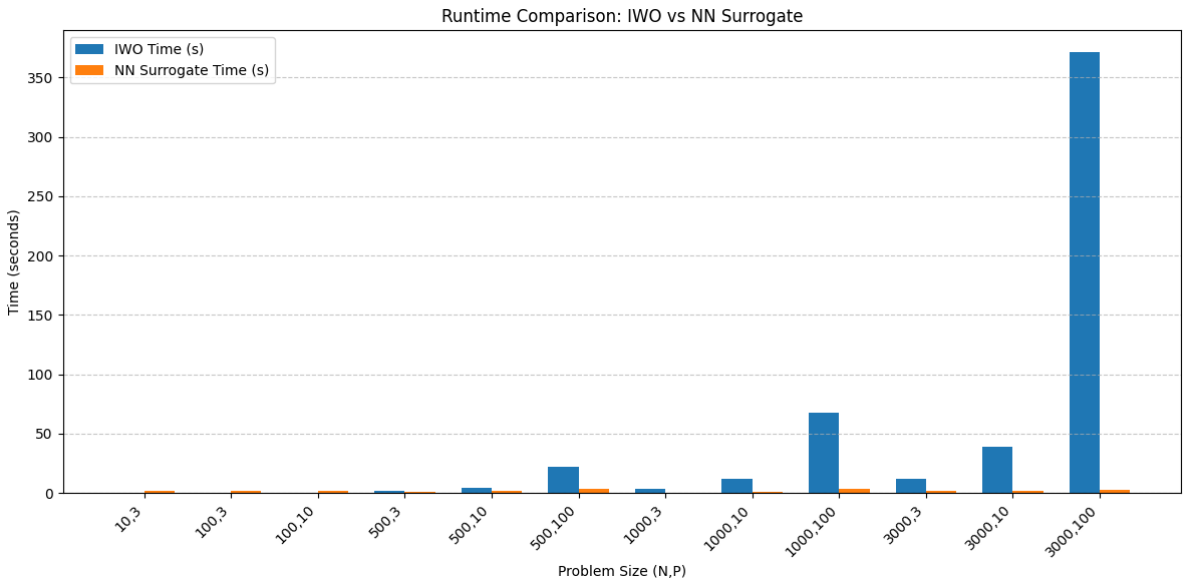
[1] Invasive Weed Optimization (IWO) is a nature-inspired metaheuristic algorithm modeled after the colonizing behavior of weeds. It simulates the way weeds adapt, reproduce, and compete in an environment to survive and spread. IWO starts with a population of candidate solutions (weeds) that generate seeds based on their fitness — better solutions produce more seeds. These seeds spread around parent locations with a Gaussian distribution whose standard deviation decreases over time, enabling the algorithm to balance exploration and exploitation. Throughout iterations, the population is selectively reduced to maintain the best solutions, mimicking natural selection. This process iteratively

improves solution quality for complex optimization problems and has demonstrated effectiveness and robustness in combinatorial problems such as the P-center problem addressed in this work.

DISCUSSING THE RESULTS OF THE PROPOSED METHODOLOGY:

The experimental results demonstrate that the integration of a neural network surrogate model within the Invasive Weed Optimization (IWO) framework effectively accelerates the optimization process while maintaining competitive solution quality. Across varying problem sizes and center counts, the surrogate model consistently reduces runtime compared to pure IWO, sometimes by an order of magnitude, illustrating its computational advantage. Although the neural network's predicted fitness occasionally deviates from the true fitness — as evidenced by mean squared error (MSE), mean absolute error (MAE), and R^2 metrics — the surrogate guides the search towards high-quality solutions efficiently. The predictive accuracy improves with larger training datasets and more extensive model training, as reflected in decreasing MSE and MAE and increasing R^2 values for larger problem instances. Notably, for moderate problem scales (e.g., N=100N=100, P=10P=10), the surrogate delivers both rapid evaluation and reasonable fitness approximation, enabling substantial speedups without significant loss in solution quality. These findings highlight the practical tradeoff between surrogate model accuracy and computational efficiency, underscoring the surrogate's role as a powerful heuristic accelerator in large-scale combinatorial optimization problems.





CONCLUSION

The developed neural network surrogate model substantially enhances the efficiency of the optimization process for the large-scale P-center problem by providing rapid fitness evaluations. By approximating the computationally intensive objective function, the surrogate allows the search algorithm to explore the solution space more quickly while maintaining solution quality. The observed predictive accuracy, supported by validation metrics such as MSE, MAE, and R^2, indicates that although the surrogate fitness predictions are approximate, they successfully guide the search toward promising solutions. Training the surrogate with diverse and sufficiently large datasets further improves its predictive capabilities, demonstrating scalability across problem sizes. Overall, this surrogate-assisted optimization framework presents a practical and effective strategy for accelerating combinatorial optimization tasks without significant compromise on solution accuracy.

REFERENCES:

[1] B. Jayalakshmi and A. Singh, "Two swarm intelligence based approaches for the p-center problem," Int. J. Electric and Hybrid Vehicles, vol. x, no. x, pp. 1–20, 2016.

[2] E. Chappidi and A. Singh, "A hyper-heuristic based approach with naive Bayes classifier for the reliability p-median problem," Applied Intelligence, Springer, 2023, doi: 10.1007/s10489-023-04983

[3] A. Yurtkuran and E. Emel, "A Modified Artificial Bee Colony Algorithm for p-Center Problems," The Scientific World Journal, vol. 2014, Article ID 824196, 9 pages, 2014, doi: 10.1155/2014/824196.

[4] M. Albareda-Sambola, Y. Hinojosa, and J. Puerto, "The reliable p-median problem with at-facility service," European Journal of Operational Research, vol. 245, pp. 656–666, 2015, doi: 10.1016/j.ejor.2015.03.049.

[5] S. R. Mousavi, "Exploiting flat subspaces in local search for p-Center problem and two fault-tolerant variants," Computers & Operations Research, vol. 149, 106023, 2023, doi: 10.1016/j.cor.2022.106023.

[6] D. Chen and R. Chen, "Optimal algorithms for the a-neighbor p-center problem," European Journal of Operational Research, vol. 225, no. 1, pp. 36–43, 2013, doi: 10.1016/j.ejor.2012.09.041